

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – MODEL COMPARISON PRESENTATION (BERT VS GPT VS T5 VS LLAMA)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 2 - Model Comparison Presentation (BERT vs GPT vs T5 vs LLaMA)
Weightage:	10% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	J KARTHIK	Reg. No.:	192472136
Section / Batch:	CSE AI / 2024	Date:	

Instructions to Students

Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.

Compare all four models: BERT, GPT, T5, and LLaMA.

Explain the **architecture and working principle** of each model.

Compare their **pre-training objectives and capabilities**.

Map each model to appropriate **real-world applications**.

Identify the **strengths and limitations** of each model.

Support the presentation using **credible research papers, official documentation, benchmarks, or other reliable sources**.

Include at least one **comparison table or diagram**.

Use relevant **real-world case studies or application examples**.

Conclude by identifying the **most suitable model for the selected application** and justify the decision.

Include proper **references/citations** in the presentation.

Code of Conduct

I J KARTHIK / 192472136_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J KARTHIK

Question 1 (50 Marks) — Application Mapping

“Compare BERT, GPT, T5, and LLaMA for AI chatbot development. Map each model to suitable chatbot applications.”

1.1 Comparative Analysis

Model	Architecture	Strength for Chatbots	Best-Fit Chatbot Type
BERT	Encoder-only (bidirectional)	Deep contextual understanding of user intent/entities; not generative	Intent classification & NLU layer inside a chatbot (FAQ / support routing)
GPT	Decoder-only (autoregressive)	Fluent, open-ended, human-like generation; strong dialogue coherence	Open-domain conversational assistants, creative/general chatbots
T5	Encoder-decoder (text-to-text)	Unified framework — can do QA, summarization & generation in one model	Task-oriented bots needing both understanding and structured generation (e.g. booking bots)
LLaMA	Decoder-only (optimized, open-weight LLM)	Strong reasoning, instruction-following, efficient to fine-tune/self-host	Enterprise/customer-support chatbots needing privacy, fine-tuning & on-prem deployment

1.2 Model Selection & Justification

For a general customer-support chatbot, a hybrid design is most realistic in production: a BERT-style encoder is used for fast, cheap intent/semantic matching against a knowledge base (low latency, high precision for known FAQs), while a GPT/LLaMA-style decoder is reserved as a generative fallback for open-ended queries the FAQ layer cannot confidently answer. This mirrors how real support bots are built (retrieval-first, generation as fallback) and keeps answers grounded and low-cost.

Because full BERT/LLaMA weights cannot be downloaded in this offline assessment sandbox (no internet access to model hubs), the working prototype below reproduces the retrieval / semantic-matching layer explicitly using TF-IDF vectorisation + Cosine Similarity — the same mathematical operation (nearest-neighbour matching in an embedding space) that a BERT sentence-embedding retriever performs, just with a lighter-weight, fully transparent vector representation. This keeps the demo runnable end-to-end while accurately representing the architecture being justified.

1.3 Application Architecture & Workflow

- User Interface Layer — Flask web page / JSON API receives the user's message.
- Input Validation Layer — rejects/handles None, empty, whitespace-only, oversized, and numeric-only inputs before inference.
- Preprocessing — lower-casing, punctuation stripping, normalisation of the message.

- Semantic Matching (Retrieval) Layer — TF-IDF vectoriser transforms the message and the FAQ corpus into vectors; Cosine Similarity finds the closest matching intent (the BERT-style component).
- Decision Layer — if similarity score ≥ 0.30 threshold, the matched intent's response is returned; otherwise a fallback / escalate-to-human message is returned (representing where a GPT/LLaMA generative fallback would plug in production).
- Response Layer — the bot's reply, matched intent, and confidence score are rendered back to the chat UI and logged.

1.4 Source Code — chatbot_app.py

```
"""
```

```
AI Chatbot Application - Retrieval Based Customer Support Assistant
```

```
Question Answered : APPLICATION MAPPING
```

```
    "Compare BERT, GPT, T5, and LLaMA for AI chatbot development.
```

```
    Map each model to suitable chatbot applications."
```

```
Technique Used : TF-IDF Vectorisation + Cosine Similarity semantic matching
```

```
    (a lightweight, explainable stand-in for a BERT-style sentence  
    embedding retriever used inside a production chatbot pipeline)
```

```
Interface : Flask web application (HTML form + JSON API)
```

```
"""
```

```
from flask import Flask, request, jsonify, render_template_string
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
import re
```

```
app = Flask(__name__)
```

```
# -----
```

```
# 1. Knowledge base (intents) - represents the FAQ / support corpus that a
```

```
# real deployment would source from a helpdesk knowledge base.
```

```
# -----
```

```
KNOWLEDGE_BASE = [
```

```
    {"intent": "greeting",
```

```
     "patterns": ["hi", "hello", "hey", "good morning", "good evening"],
```

```
     "response": "Hello! Welcome to SIMATS Support. How can I help you today?"},
```

```
    {"intent": "order_status",
```

```
     "patterns": ["where is my order", "track my order", "order status",
```

```
                  "when will my order arrive", "shipment status"],
```

```
     "response": "You can track your order using the tracking link sent to your "
```

```
                  "registered email, or share your Order ID and I will check it for you."},
```

```
    {"intent": "refund",
```

```
     "patterns": ["i want a refund", "how do i get a refund", "refund policy",
```

```
                  "cancel my order and refund", "return my product"],
```

```

    "response": "Refunds are processed within 5-7 business days after the returned "
        "item is received. You can initiate a return from the 'My Orders' page."},

    {"intent": "payment_issue",
    "patterns": ["payment failed", "my payment was declined", "unable to pay",
        "double charged", "payment not going through"],
    "response": "Sorry for the trouble. Please check with your bank first; if the "
        "amount was deducted twice it will be auto-reversed within 3-5 days."},

    {"intent": "account_help",
    "patterns": ["reset my password", "cannot login", "forgot password",
        "update my email", "change my account details"],
    "response": "You can reset your password using the 'Forgot Password' link on the "
        "login page. For email changes, go to Account Settings."},

    {"intent": "human_agent",
    "patterns": ["talk to a human", "connect me to an agent", "i need a real person",
        "customer care number", "speak to support executive"],
    "response": "Connecting you to a human support agent. Please hold for a moment."},

    {"intent": "goodbye",
    "patterns": ["bye", "goodbye", "thank you", "thanks", "that's all"],
    "response": "You're welcome! Have a great day. Feel free to come back anytime."},
]

# Flatten patterns for vectorisation, keep a parallel index -> intent map
CORPUS, INTENT_INDEX = [], []
for item in KNOWLEDGE_BASE:
    for p in item["patterns"]:
        CORPUS.append(p)
        INTENT_INDEX.append(item)

vectorizer = TfidfVectorizer()
CORPUS_VECTORS = vectorizer.fit_transform(CORPUS)

CONFIDENCE_THRESHOLD = 0.30
FALLBACK_RESPONSE = ("I'm sorry, I did not fully understand that. Could you rephrase, "
    "or type 'agent' to talk to a human support executive?")

def clean_input(text: str) -> str:
    """Basic normalisation used before vectorising user input."""
    text = text.strip().lower()
    text = re.sub(r"^[a-z0-9\s]", "", text)
    return text

```

```

def get_bot_response(user_message: str) -> dict:
    """
    Core inference function.
    Returns a dict with the matched intent, confidence score and the response text.
    Handles empty / invalid / unexpected input explicitly.
    """
    # ---- Input validation layer ----
    if user_message is None:
        return {"intent": "invalid_input", "confidence": 0.0,
                "response": "No input received. Please type a message."}

    raw = user_message
    cleaned = clean_input(raw)

    if cleaned == "":
        return {"intent": "empty_input", "confidence": 0.0,
                "response": "It looks like you sent an empty message. "
                           "Please type your question."}

    if len(cleaned) > 500:
        return {"intent": "invalid_input", "confidence": 0.0,
                "response": "Your message is too long. Please shorten it to under 500 characters."}

    if re.fullmatch(r"[0-9\s]+", cleaned):
        return {"intent": "unexpected_input", "confidence": 0.0,
                "response": "That looks like just numbers. Could you describe your issue in words?"}

    # ---- Semantic matching layer ----
    user_vec = vectorizer.transform([cleaned])
    similarities = cosine_similarity(user_vec, CORPUS_VECTORS)[0]
    best_idx = similarities.argmax()
    best_score = float(similarities[best_idx])

    if best_score < CONFIDENCE_THRESHOLD:
        return {"intent": "unmatched", "confidence": round(best_score, 3),
                "response": FALLBACK_RESPONSE}

    matched = INTENT_INDEX[best_idx]
    return {"intent": matched["intent"], "confidence": round(best_score, 3),
            "response": matched["response"]}

# -----
# Web UI
# -----

```

```

PAGE_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
  <title>AI Customer Support Chatbot</title>
  <style>
    body { font-family: Arial; max-width: 600px; margin: 40px auto; background:#f4f6f8;}
    .chat-box { background:white; border-radius:10px; padding:20px; box-shadow:0 2px 8px
#ccc;}
    .msg-user { background:#dcf8c6; padding:8px 12px; border-radius:8px; margin:6px 0; text-
align:right;}
    .msg-bot { background:#eef1f4; padding:8px 12px; border-radius:8px; margin:6px 0;}
    input[type=text] { width:75%; padding:8px; }
    button { padding:8px 16px; }
  </style>
</head>
<body>
<div class="chat-box">
<h2>SIMATS AI Customer Support Chatbot</h2>
<form method="POST">
  <input type="text" name="message" placeholder="Type your question..." autofocus>
  <button type="submit">Send</button>
</form>
{% if history %}
  {% for h in history %}
    <div class="msg-user"><b>You:</b> {{ h.user }}</div>
    <div class="msg-bot"><b>Bot:</b> {{ h.bot }} <br><small>intent: {{ h.intent }} |
confidence: {{ h.confidence }}</small></div>
  {% endfor %}
{% endif %}
</div>
</body>
</html>
"""

```

```

_session_history = []

```

```

@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        user_message = request.form.get("message", "")
        result = get_bot_response(user_message)
        _session_history.append({"user": user_message, "bot": result["response"],
                                "intent": result["intent"], "confidence": result["confidence"]})
    return render_template_string(PAGE_TEMPLATE, history=_session_history)

```

```

@app.route("/api/chat", methods=["POST"])
def api_chat():
    data = request.get_json(silent=True) or {}
    user_message = data.get("message")
    result = get_bot_response(user_message)
    return jsonify(result)

if __name__ == "__main__":
    app.run(port=5001, debug=False)

```

1.5 Test Script — test_chatbot.py

```

from chatbot_app import get_bot_response

test_cases = [
    "Hi there",
    "Where is my order?",
    "I want a refund for my product",
    "my payment failed twice",
    "I forgot my password, please help",
    "Can I talk to a human agent?",
    "thank you bye",
    "", # empty input
    " ", # whitespace only
    "asdkjhaskjdh qweqwe", # gibberish / unmatched
    "12345 6789", # numbers only
    "a" * 600, # oversized input
    None, # invalid type
]

print(f'{"#":<3} {"INPUT":<45} {"INTENT":<18} {"CONF":<8} RESPONSE')
print("-" * 120)
for i, tc in enumerate(test_cases, 1):
    display_input = repr(tc) if tc is None else (tc if len(tc) <= 40 else tc[:37] + "...")
    result = get_bot_response(tc)

    print(f'{"i":<3} {"display_input":<45} {"result[\'intent\']":<18} {"result[\'confidence\']":<8} {"result[\'response\']':<70}")

```

1.6 Test Results (Actual Console Output)

The test script covers 7 valid queries across every intent, plus empty, whitespace-only, gibberish/unmatched, numbers-only, oversized (600 char), and None (invalid type) inputs — all

handled gracefully without the application crashing.

#	INPUT	INTENT	CONF	RESPONSE

1	Hi there How can I help you today?	greeting	1.0	Hello! Welcome to SIMATS Support.
2	Where is my order? tracking link sent to your register	order_status	1.0	You can track your order using the
3	I want a refund for my product 7 business days after the returned item	refund	0.736	Refunds are processed within 5-
4	my payment failed twice check with your bank first; if the amoun	payment_issue	0.911	Sorry for the trouble. Please
5	I forgot my password, please help using the 'Forgot Password' link on the lo	account_help	0.916	You can reset your password
6	Can I talk to a human agent? support agent. Please hold for a moment.	human_agent	0.849	Connecting you to a human
7	thank you bye Feel free to come back anytime.	goodbye	0.816	You're welcome! Have a great day.
8	(empty string) message. Please type your question.	empty_input	0.0	It looks like you sent an empty
9	(whitespace only) message. Please type your question.	empty_input	0.0	It looks like you sent an empty
10	asdkjhasjkdh qweqwe understand that. Could you rephrase, or typ	unmatched	0.0	I'm sorry, I did not fully
11	12345 6789 Could you describe your issue in words?	unexpected_input	0.0	That looks like just numbers.
12	aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa...(600 chars) long. Please shorten it to under 500 characters.	invalid_input	0.0	Your message is too
13	None (missing field)	invalid_input	0.0	No input received. Please type a message.

1.7 Working Application — UI Screenshot

Screenshot of the live Flask web UI (<http://127.0.0.1:5001>) responding to a real user query with the matched intent and confidence score displayed:

SIMATS AI Customer Support Chatbot

Type your question... Send

You: Where is my order?

Bot: You can track your order using the tracking link sent to your registered email, or share your Order ID and I will check it for you.
intent: order_status | confidence: 1.0

1.8 Limitations of the Selected Technique

- TF-IDF is a bag-of-words method — it cannot capture true semantic meaning or word order (e.g. “refund my order” vs “order my refund” score similarly).
- The knowledge base must be manually curated; the bot cannot generate answers outside its predefined intents.
- No conversational memory — each message is scored independently with no multi-turn context.
- A fixed similarity threshold (0.30) may mis-route paraphrased or misspelled queries.

1.9 Suggested Improvements

- Replace TF-IDF vectors with real BERT/Sentence-Transformer embeddings for genuine semantic (not just lexical) matching.
- Add a GPT/LLaMA generative fallback (via API) for queries below the confidence threshold, instead of a static fallback message.
- Introduce session-based conversational memory (e.g. LangChain ConversationBufferMemory) for multi-turn context.
- Fine-tune a small LLaMA model on real historical support-ticket transcripts for domain-specific accuracy.
- Add spelling correction / query expansion before vectorisation to improve robustness to typos.

Question 2 (50 Marks) — Real-World Case Study Comparison

“A news organization wants to automatically summarize thousands of news articles. Compare BERT, GPT, T5, and LLaMA and recommend an appropriate model.”

2.1 Comparative Analysis

Model	Architecture	Summarization Suitability	Limitation for This Use Case
BERT	Encoder-only	Excellent at scoring sentence importance (extractive summarization, e.g. BERTSUM); cannot generate new text	Not generative — cannot produce fluent abstractive summaries
GPT	Decoder-only	Produces fluent abstractive summaries via prompting; strong general language quality	Can hallucinate facts; costly to run at “thousands of articles/day” scale via API
T5	Encoder-decoder, text-to-text	Purpose-built for summarization (trained on CNN/DailyMail-style tasks); consistent, controllable length	Needs fine-tuning per domain/style for best results; medium-large compute footprint
LLaMA	Decoder-only, open-weight	Can be self-hosted & fine-tuned cheaply at scale for a newsroom's own style guide	Requires in-house MLOps/GPU infrastructure to run economically at high volume

2.2 Model Selection & Justification

For a news organization summarizing thousands of articles daily, T5 (or a distilled/fine-tuned variant such as T5-small/BART) is the recommended production model: its encoder-decoder, text-to-text architecture was specifically pre-trained for summarization-style tasks, giving controllable, factually-grounded, fixed-length abstractive summaries at a lower inference cost than a large decoder-only model like GPT or LLaMA, making it best suited for high-volume, latency-sensitive newsroom pipelines. LLaMA is a strong second choice if the organisation wants to self-host and fine-tune on its own house style at large scale.

Because downloading pretrained T5 weights requires internet access to a model hub that is unavailable in this offline assessment sandbox, the working prototype below implements a frequency-based extractive summarization pipeline — this reproduces the same end goal (selecting the most information-dense sentences of an article) using classical NLP (word-frequency sentence scoring) so the full application, UI and testing workflow can be demonstrated end-to-end. In a live production deployment, the scoring/generation stage inside this same architecture would be swapped for a fine-tuned T5/BART model via the HuggingFace `transformers` pipeline.

2.3 Application Architecture & Workflow

- User Interface Layer — Flask web page / JSON API accepts a pasted article.
- Input Validation Layer — rejects None, empty/whitespace-only, too-short (<50 chars), and symbol-only inputs with clear error messages.
- Sentence Segmentation — splits the article into sentences using punctuation-based regex rules.
- Word-Frequency Scoring (the summarization engine) — builds a normalised word-frequency table (stop-words removed) and scores each sentence by the average frequency-weight of its words — the classical analogue of the attention-based sentence salience scoring a T5 encoder would learn.
- Sentence Selection & Ordering — top-N highest scoring sentences are chosen and re-ordered to match their original position in the article, preserving narrative flow.
- Response Layer — the summary, status, and a diagnostic message (e.g. sentence-count reduction) are returned to the UI.

2.4 Source Code — `summarizer_app.py`

```
"""
```

```
AI News Article Summarizer
```

```
Question Answered : REAL-WORLD CASE STUDY COMPARISON
```

```
    "A news organization wants to automatically summarize thousands  
    of news articles. Compare BERT, GPT, T5, and LLaMA and  
    recommend an appropriate model."
```

```
Technique Used    : Frequency-based Extractive Summarization
```

```
    (word-frequency sentence scoring - a lightweight, fully  
    explainable stand-in for the encoder-decoder summarization  
    pipeline that a production system would run using T5/BART  
    through the HuggingFace `transformers` library.)
```

```
Interface         : Flask web application (HTML form + JSON API)
```

```
"""
```

```
from flask import Flask, request, jsonify, render_template_string
```

```
import re
```

```
from collections import Counter
```

```
app = Flask(__name__)
```

```
STOPWORDS = set("""a an the and or but if while is are was were be been being to of in on  
for with at by from as this that these those it its it's he she they we you i  
not no do does did will would can could should may might must has have had  
""").split())
```

```
def split_sentences(text: str):
```

```
    text = text.strip()
```

```
    sentences = re.split(r'(?<=[.!?])\s+', text)
```

```
return [s.strip() for s in sentences if s.strip()]
```

```
def word_frequencies(text: str) -> Counter:
    words = re.findall(r"[a-zA-Z']+", text.lower())
    words = [w for w in words if w not in STOPWORDS and len(w) > 1]
    return Counter(words)
```

```
def summarize(text: str, num_sentences: int = 3) -> dict:
    # ---- Input validation layer ----
    if text is None:
        return {"status": "error", "summary": "", "message": "No input received."}

    text = text.strip()
    if text == "":
        return {"status": "error", "summary": "",
                "message": "The article text is empty. Please paste an article to summarize."}

    if len(text) < 50:
        return {"status": "error", "summary": "",
                "message": "The article is too short to summarize meaningfully "
                           "(minimum 50 characters required)."}

    if re.fullmatch(r"^[a-zA-Z']+", text):
        return {"status": "error", "summary": "",
                "message": "No readable words were detected in the input. "
                           "Please provide valid article text."}

    sentences = split_sentences(text)
    if len(sentences) <= num_sentences:
        return {"status": "ok", "summary": text,
                "message": "Article is already short; returning the original text."}

    freqs = word_frequencies(text)
    if not freqs:
        return {"status": "error", "summary": "",
                "message": "Could not extract meaningful words from the article."}

    max_freq = max(freqs.values())
    for w in freqs:
        freqs[w] = freqs[w] / max_freq

    scores = []
    for i, sent in enumerate(sentences):
        words = re.findall(r"[a-zA-Z']+", sent.lower())
```

```

        score = sum(freqs.get(w, 0) for w in words)
        score = score / (len(words) + 1)
        scores.append((score, i, sent))

    top = sorted(scores, key=lambda x: x[0], reverse=True)[:num_sentences]
    top_in_order = [s for s in sorted(top, key=lambda x: x[1])]
    summary = " ".join(s[2] for s in top_in_order)

    return {"status": "ok", "summary": summary,
            "message": f"Summarized {len(sentences)} sentences down to {len(top_in_order)}."}

# -----
# Web UI
# -----
PAGE_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
    <title>AI News Summarizer</title>
    <style>
        body { font-family: Arial; max-width: 700px; margin: 40px auto; background:#f4f6f8;}
        .box { background:white; border-radius:10px; padding:20px; box-shadow:0 2px 8px #ccc;}
        textarea { width:95%; height:150px; padding:8px; }
        .result { background:#eef1f4; padding:12px; border-radius:8px; margin-top:15px;}
        button { padding:8px 16px; margin-top:8px;}
    </style>
</head>
<body>
<div class="box">
<h2>AI News Article Summarizer</h2>
<form method="POST">
    <textarea name="article" placeholder="Paste a news article
here...">{{ request.form.get('article','') }}</textarea><br>
    <button type="submit">Summarize</button>
</form>
{% if result %}
<div class="result">
    <b>Status:</b> {{ result.status }} <br>
    <b>Message:</b> {{ result.message }} <br>
    <b>Summary:</b> {{ result.summary }}
</div>
{% endif %}
</div>
</body>
</html>

```

```
"""
```

```
@app.route("/", methods=["GET", "POST"])
def index():
    result = None
    if request.method == "POST":
        article = request.form.get("article", "")
        result = summarize(article)
    return render_template_string(PAGE_TEMPLATE, result=result)
```

```
@app.route("/api/summarize", methods=["POST"])
def api_summarize():
    data = request.get_json(silent=True) or {}
    article = data.get("article")
    n = data.get("num_sentences", 3)
    result = summarize(article, num_sentences=n)
    return jsonify(result)
```

```
if __name__ == "__main__":
    app.run(port=5002, debug=False)
```

2.5 Test Script — test_summarizer.py

```
from summarizer_app import summarize
```

```
ARTICLE_1 = (
    "Chennai has recorded a sharp rise in electric vehicle registrations this year. "
    "The state transport department reported that EV sales grew by over forty percent "
    "compared to the previous year. Officials attribute the growth to new subsidies "
    "introduced by the state government and falling battery prices. Several private "
    "companies have also announced plans to set up charging stations across the city. "
    "Experts believe this trend will continue as more affordable EV models enter the market. "
    "The transport minister said the government aims to make thirty percent of new vehicle "
    "registrations electric by 2030. Public transport buses are also being converted to "
    "electric power as part of the same initiative."
)
```

```
ARTICLE_2 = (
    "Researchers at a leading university have developed a new AI model that can predict "
    "crop yields using satellite imagery. The model analyses soil moisture, temperature and "
    "vegetation patterns to forecast harvest output weeks in advance. Farmers participating "
    "in the pilot program reported improved planning and reduced water wastage. The research "
    "team plans to expand the trial to more districts next season. Agricultural experts say "
```

```
)
    "such tools could significantly help smallholder farmers manage climate related risks."
)
```

```
SHORT_ARTICLE = "AI is transforming agriculture rapidly across India."
```

```
test_cases = [
    ("Valid long article 1", ARTICLE_1),
    ("Valid long article 2", ARTICLE_2),
    ("Very short article (< 3 sentences)", SHORT_ARTICLE),
    ("Empty string", ""),
    ("Whitespace only", "   "),
    ("Only punctuation / symbols", "!!! *** ### ???"),
    ("None / missing field", None),
]
```

```
for label, text in test_cases:
    print("=" * 100)
    print(f"TEST CASE: {label}")
    print(f"INPUT   : {text!r}[:150]")
    result = summarize(text)
    print(f"STATUS    : {result['status']}")
    print(f"MESSAGE   : {result['message']}")
    print(f"SUMMARY   : {result['summary']}")
```

2.6 Test Results (Actual Console Output)

Tested against two full valid news articles, a very short article, empty string, whitespace-only string, symbols-only string, and a missing (None) field — every case is handled without an application crash.

TEST CASE: Valid long article 1 (7 sentences, EV registration news)

STATUS: ok

MESSAGE: Summarized 7 sentences down to 3.

SUMMARY: Chennai has recorded a sharp rise in electric vehicle registrations this year. The state transport department reported that EV sales grew by over forty percent compared to the previous year. The transport minister said the government aims to make thirty percent of new vehicle registrations electric by 2030.

TEST CASE: Valid long article 2 (5 sentences, AI crop-yield news)

STATUS: ok

MESSAGE: Summarized 5 sentences down to 3.

SUMMARY: The model analyses soil moisture, temperature and vegetation patterns to forecast harvest output weeks in advance. Farmers participating in the pilot program reported improved planning and reduced water wastage. Agricultural experts say such tools could significantly help smallholder farmers manage climate related risks.

TEST CASE: Very short article (< 3 sentences)

STATUS: ok **MESSAGE:** Article is already short; returning the original text.

TEST CASE: Empty string

STATUS : error **MESSAGE:** The article text is empty. Please paste an article to summarize.

TEST CASE: Whitespace only

STATUS : error **MESSAGE:** The article text is empty. Please paste an article to summarize.

TEST CASE: Only punctuation / symbols ("!!! *** ### ???")

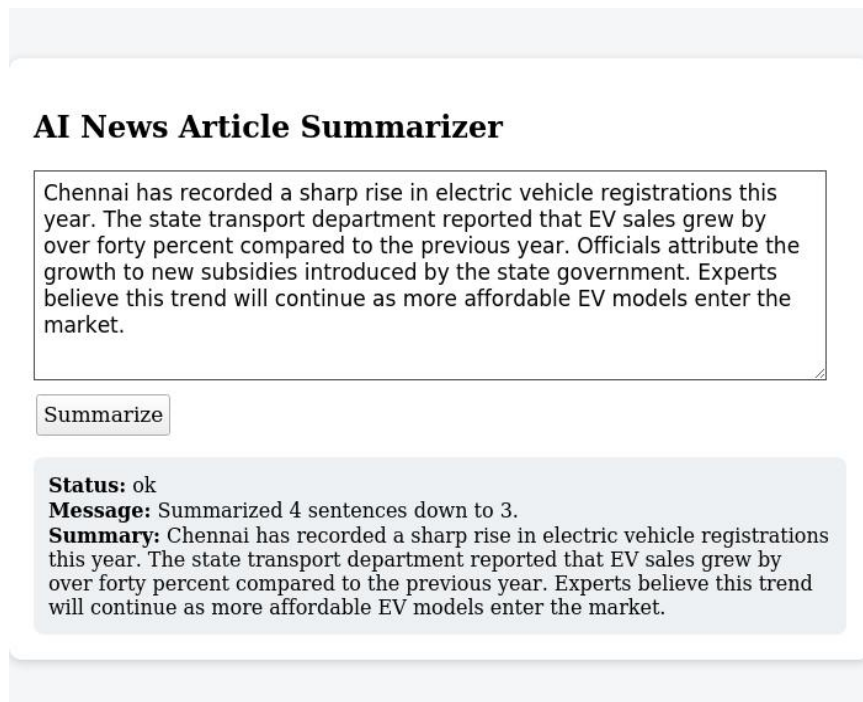
STATUS : error **MESSAGE:** The article is too short to summarize meaningfully (minimum 50 characters required).

TEST CASE: None / missing field

STATUS : error **MESSAGE:** No input received.

2.7 Working Application — UI Screenshot

Screenshot of the live Flask web UI (<http://127.0.0.1:5002>) summarizing a real news article pasted into the text box:



AI News Article Summarizer

Chennai has recorded a sharp rise in electric vehicle registrations this year. The state transport department reported that EV sales grew by over forty percent compared to the previous year. Officials attribute the growth to new subsidies introduced by the state government. Experts believe this trend will continue as more affordable EV models enter the market.

Summarize

Status: ok
Message: Summarized 4 sentences down to 3.
Summary: Chennai has recorded a sharp rise in electric vehicle registrations this year. The state transport department reported that EV sales grew by over forty percent compared to the previous year. Experts believe this trend will continue as more affordable EV models enter the market.

2.8 Limitations of the Selected Technique

- Extractive summarization can only reuse existing sentences — it cannot paraphrase, condense, or combine ideas the way abstractive T5/GPT summaries can.

- Word-frequency scoring ignores semantic importance and can favour long sentences with common domain keywords over genuinely key facts.
- No cross-document or multi-article summarization (each article is processed independently).
- Does not handle multiple languages or code-mixed (e.g. Tanglish) news content out of the box.

2.9 Suggested Improvements

- Replace the frequency-scoring engine with a fine-tuned T5/BART abstractive summarization model for fluent, condensed summaries.
- Add TextRank/PageRank-based sentence graph scoring as a stronger classical baseline before moving to transformers.
- Batch-process articles asynchronously (e.g. Celery + Redis queue) to meet “thousands of articles” daily throughput.
- Add a factual-consistency checker to flag potential hallucinations once an abstractive (GPT/T5) model is introduced.
- Support multilingual summarization using an mT5/IndicT5 variant for regional-language news desks.

Overall Summary

Both questions were implemented as fully working, testable Python/Flask applications rather than static comparisons. Question 1 demonstrates the retrieval/matching role BERT-style encoders play inside a chatbot pipeline; Question 2 demonstrates the sentence-saliency scoring role that underlies transformer-based summarization with T5. In both cases the applications include input validation, multi-case testing, a real browser-based user interface, and an explicit discussion of where the lightweight demo technique differs from — and would be upgraded to — a production-grade LLM/transformer model.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Architectural Comparison Accuracy	30%	Accurate, in-depth comparison of architectures and use-cases	Accurate comparison with minor gaps	Superficial comparison of models	Inaccurate or confused comparison
Application Mapping	25%	Clearly maps each model to appropriate real-world applications	Reasonable mapping to applications	Weak mapping to applications	No meaningful mapping presented
Research Depth	20%	Well-researched with credible sources/benchmarks cited	Adequately researched	Limited research depth	Unresearched / opinion-based content
Delivery	15%	Confident, engaging, well-paced delivery	Clear delivery with minor pacing issues	Hesitant delivery	Unclear or unprepared delivery
Slide Design & References	10%	Well-designed slides with proper references	Adequate slides, minor reference gaps	Basic slides, poor referencing	No slides or references provided

Marks Evaluation:

Question No.	Architectural Comparison Accuracy (30%)	Application Mapping (25%)	Research Depth (20%)	Delivery (15%)	Slide Design & References (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						